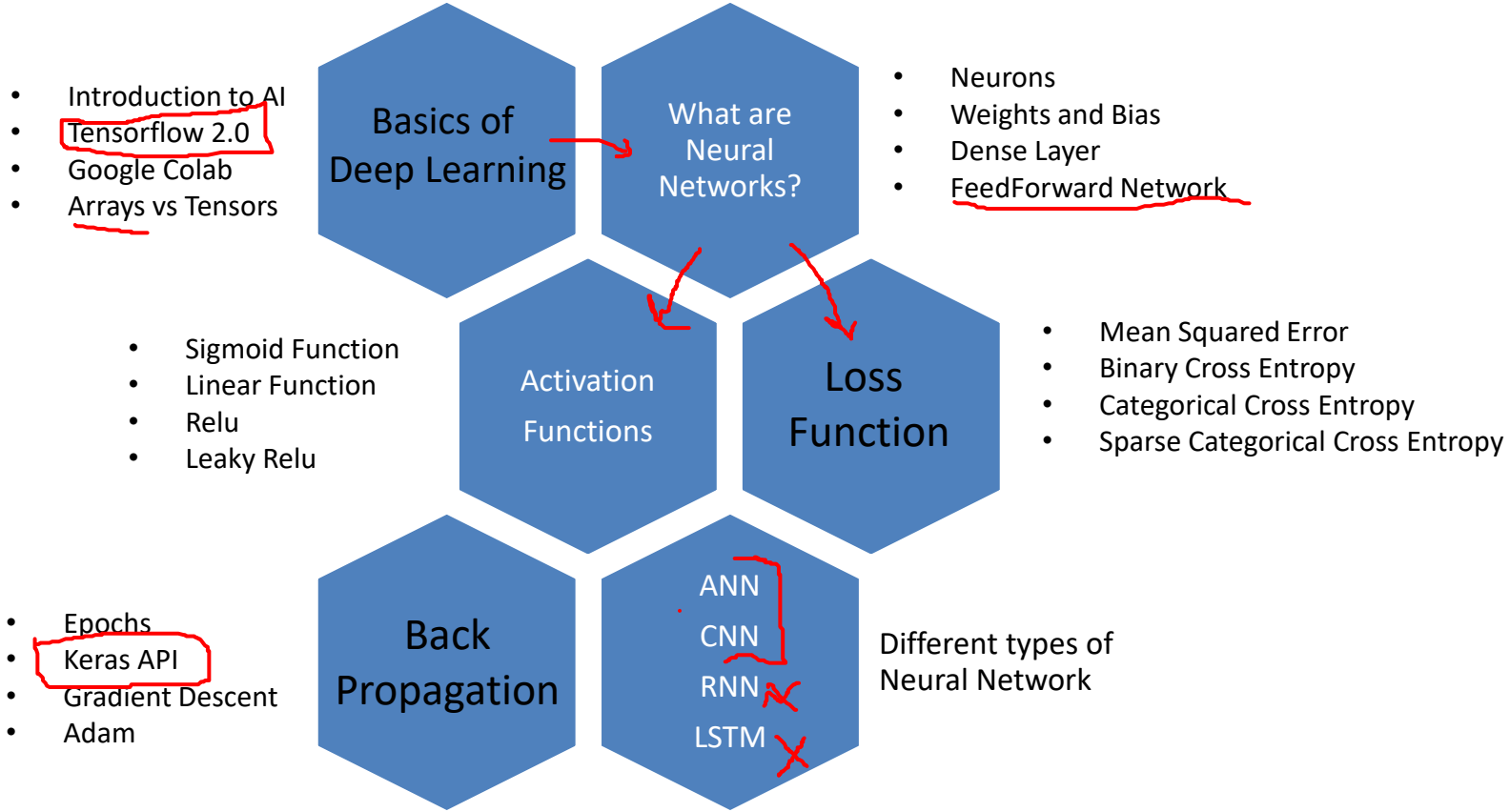
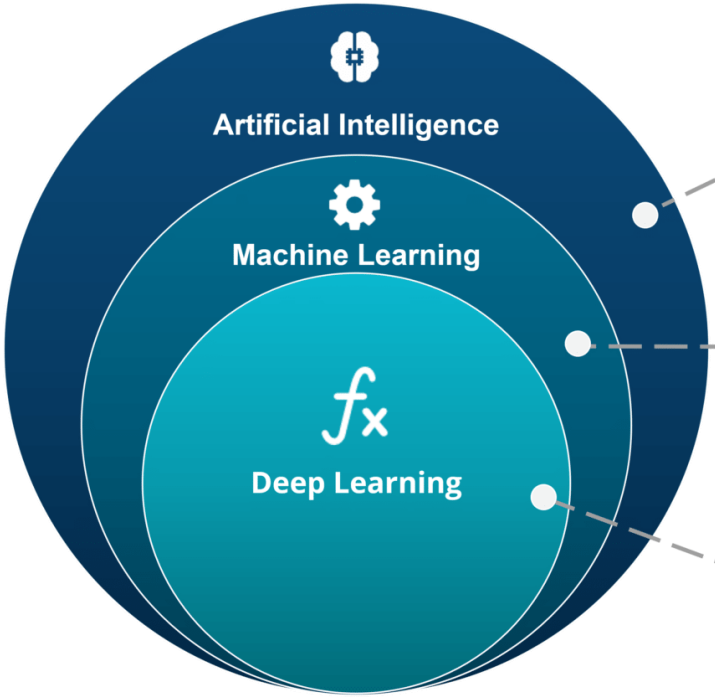
Artificial Intelligence

- Deep Learning
- Optical Character Recognition
- Facial Recognition
- Object Detection
- Chatbots
- Sentiment Analysis

Deep Learning



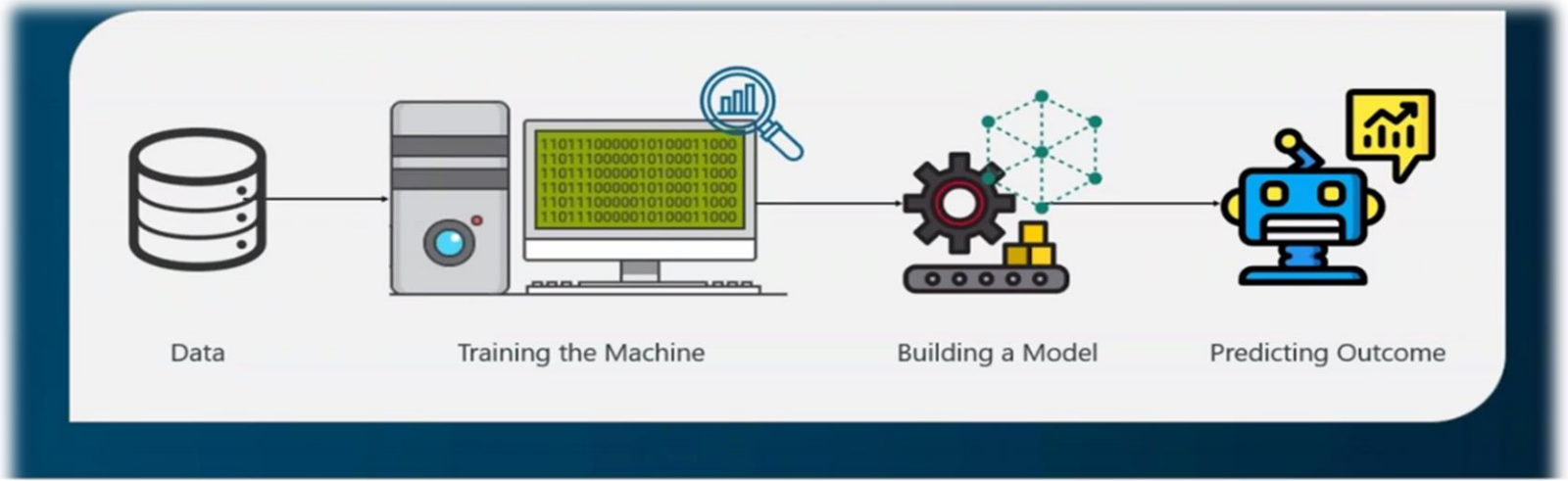
AI vs ML vs DL



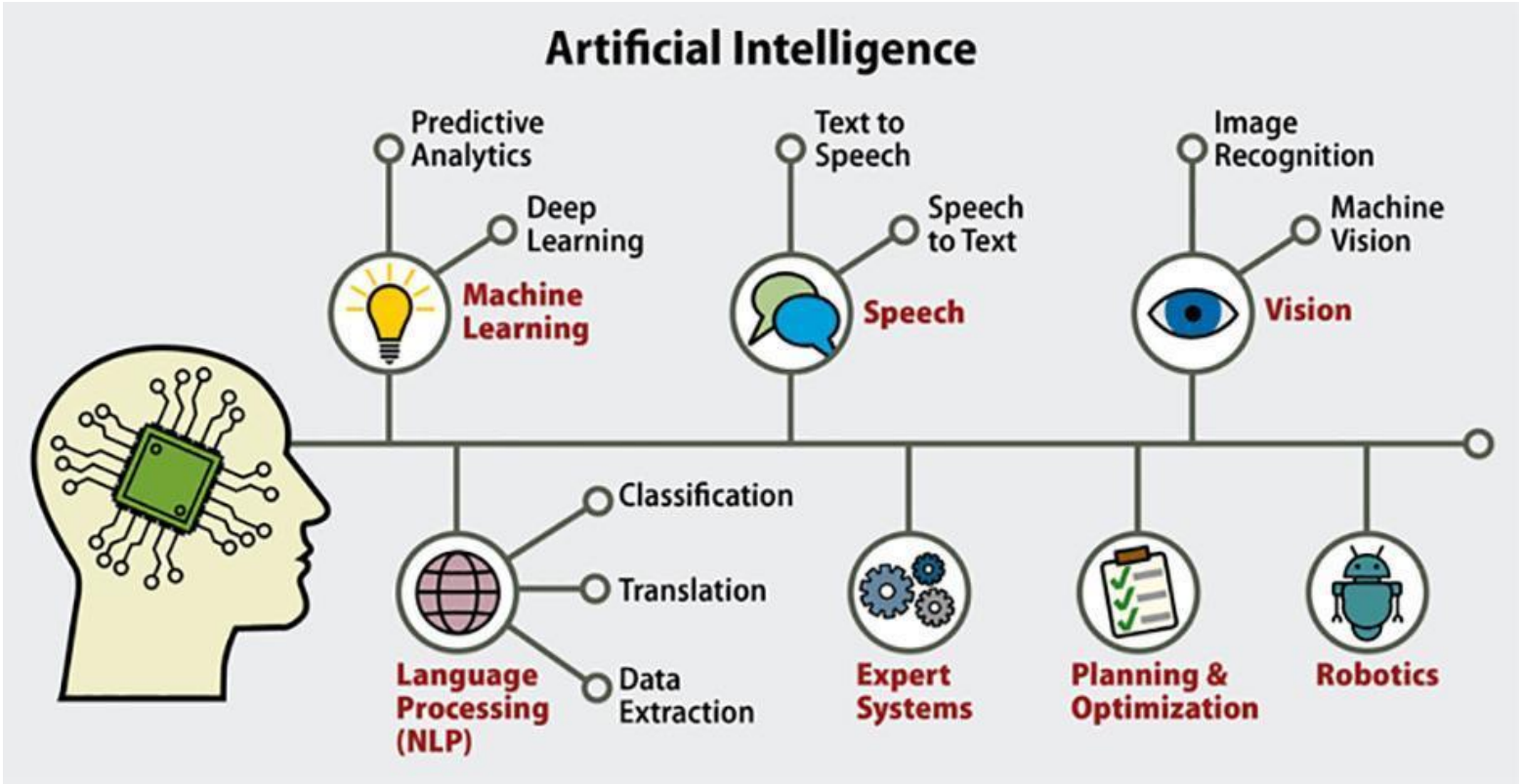
ARTIFICIAL INTELLIGENCE
A technique which enables machines to mimic human behaviour

MACHINE LEARNING
Subset of AI technique which use statistical methods to enable machines to improve with experience

DEEP LEARNING
Subset of ML which make the computation of multi-layer neural network feasible



Data can be either : 1. Numbers
2. Text
3. Image
4. Videos
5. Audios



Data can be Numbers, Images, Audio, Video or Text

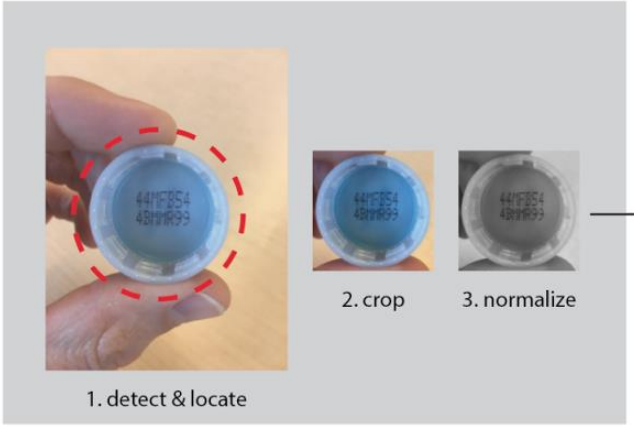
Applications of AI

- Image Recognition
- Voice Assistant / Chatbots
- Automatic Machine Translation
- Self Driving Cars
- Recommendation System

1 Take a picture

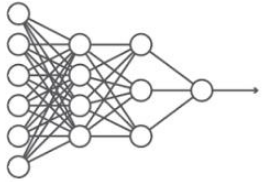


2 Process the image



3 Extract the text

TensorFlow™



A trained Neural Network
predicts the 14 pincode values

4 Display the results



Bingo!
A valid pincode was predicted



Close!
We're off by a few characters,
ask the user to correct them

send corrected code
and image back to training



FIRST CAM SET UP NEAR MAHAMAYA FLYOVER



- Radar-based cameras are installed on the sides of a road
- The radar sends out radio waves that reflect off vehicles to record their speed. If it detects traffic violations, the camera captures images of the vehicle, its numberplate and a video of the violation
- The recordings are sent to the traffic department server. They will soon be integrated into the e-challan system

Traffic violations it can detect: Speeding, jumping red lights and lanes, riding without helmet/seat belt, triple riding, talking on mobile phone, tailgating

COST: ₹20 lakh per radar camera, ₹50 lakh for software

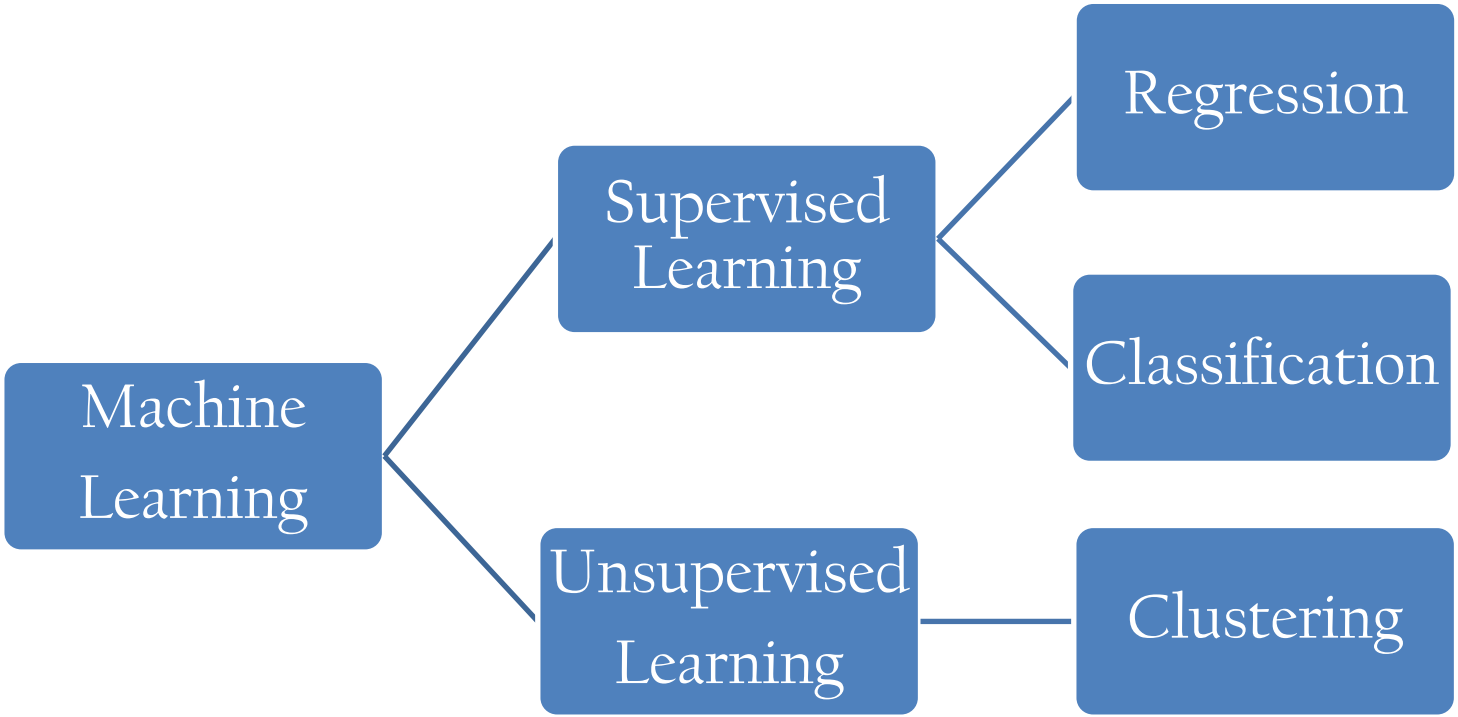


1,200 challans issued in a week for violations recorded by the system

Similar cameras have been installed along roads in Paris, Dubai and Abu Dhabi

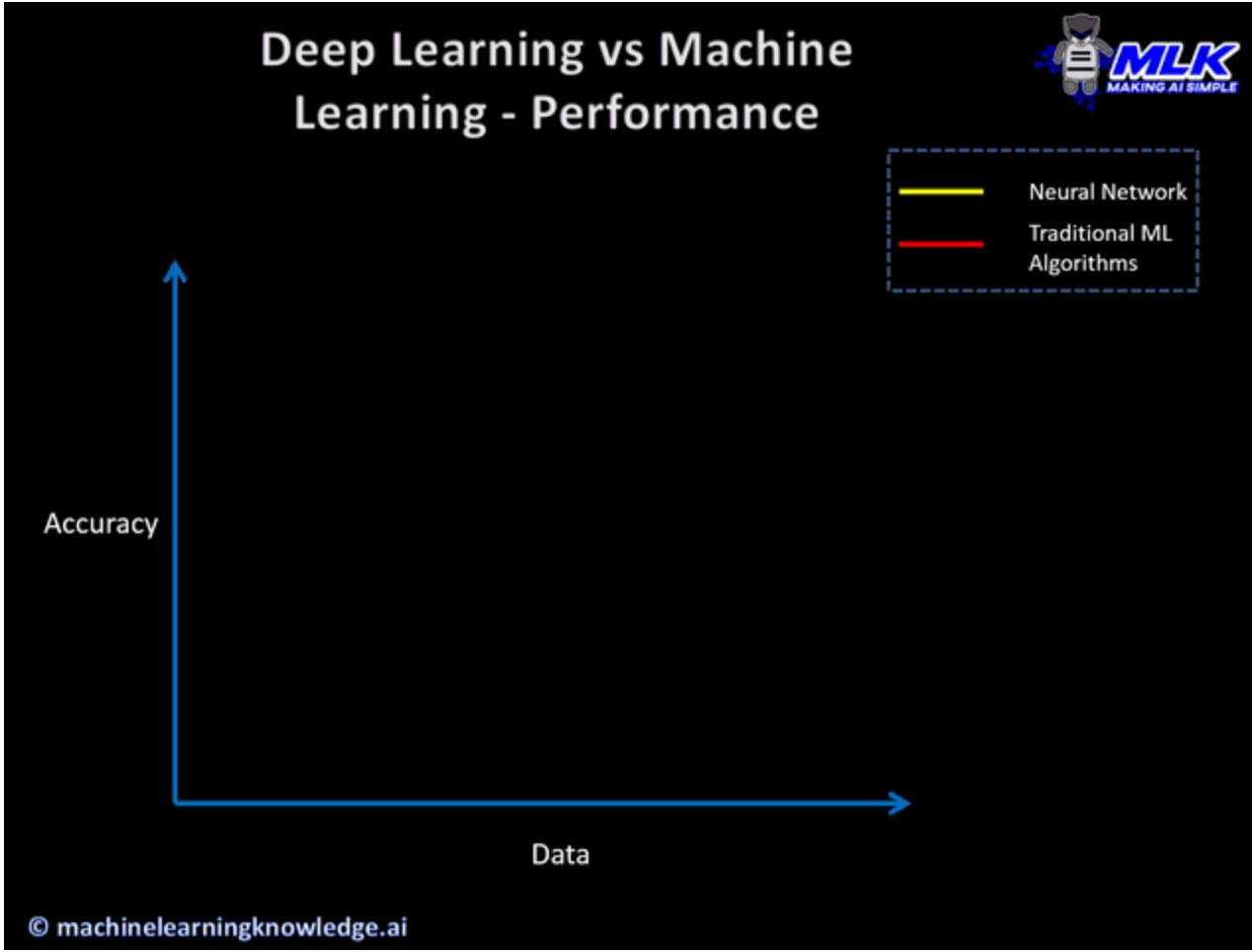
FEATURES OF THE SYSTEM:

- 98% accuracy claimed
- Flash for night vision
- Can cover up to 8 lanes and capture details of 32 vehicles at a time
- 180-degree coverage


Machine Learning
Supervised Learning
Unsupervised Learning
Regression
Classification
Clustering

Types of Data

1. Structured Data
2. Unstructured Data



Tools needed





Deep Learning Frameworks

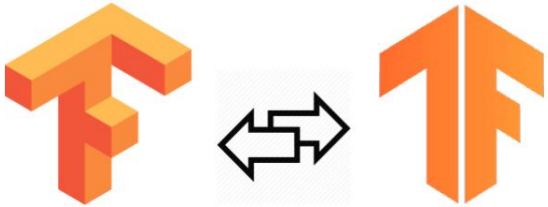












TensorFlow 2.0



Easy

Simplified APIs.
Focused on Keras and
eager execution



Powerful

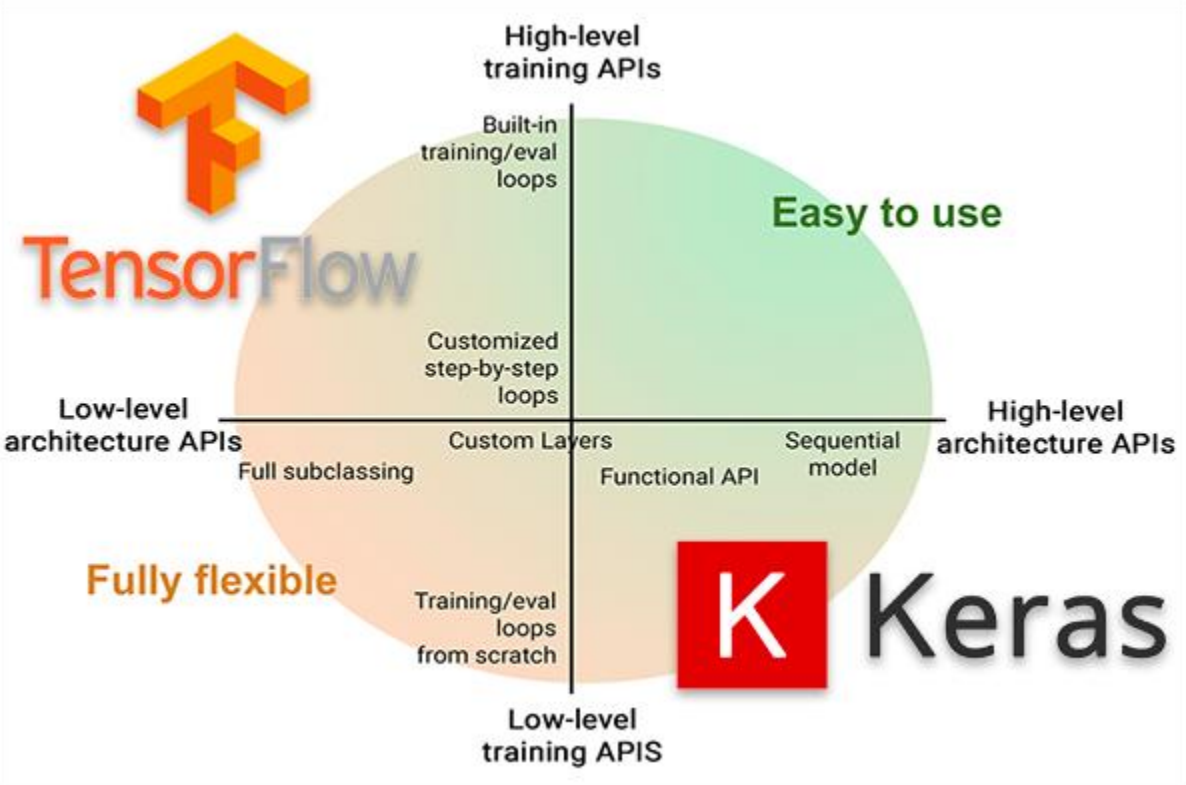
Flexibility and performance.
Power to do cutting edge research
and scale to > 1 exaflops



Scalable

Tested at Google-scale.
Deploy everywhere

Open Source Library for Graph based Numerical Computation
Developed by Google Brain Team

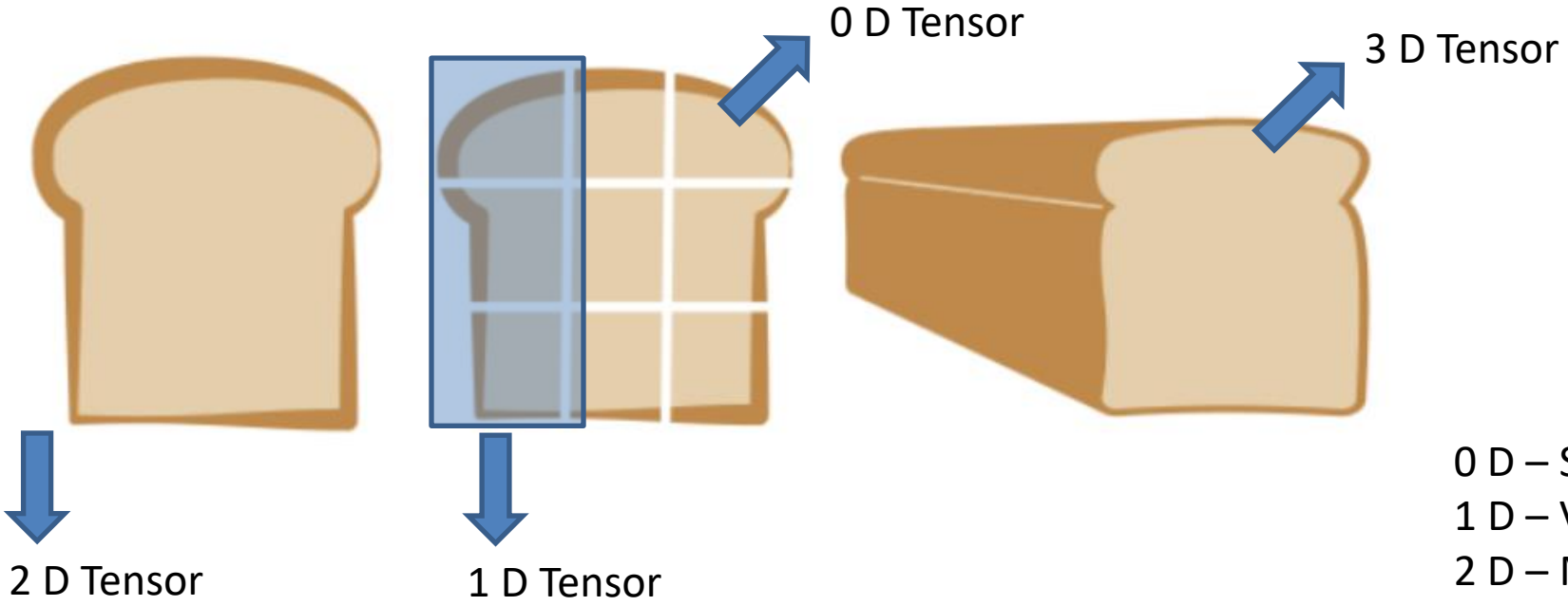


Tensorflow performs

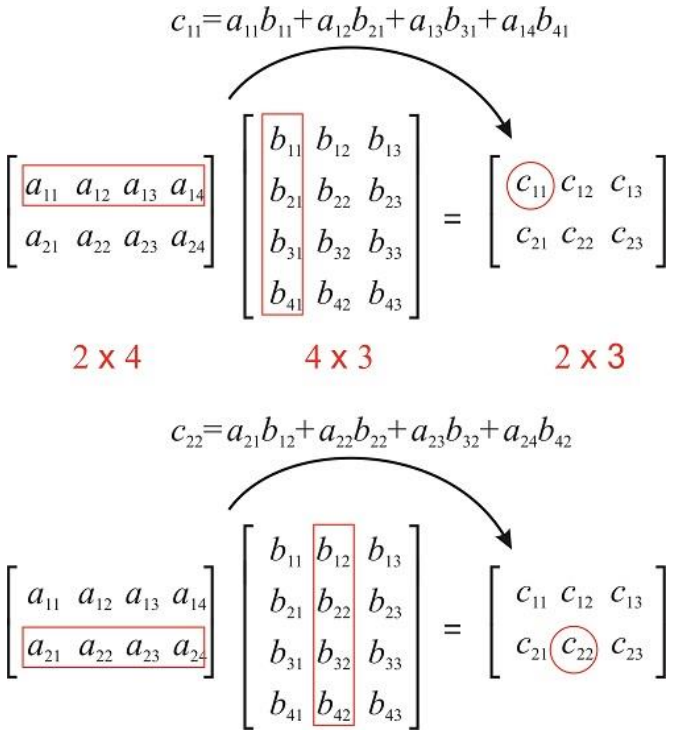
- Addition
- Multiplication
- Differentiation
- Building Neural Networks

What is a Tensor?

- Generalization of Vectors and Matrices



Difference between tf.multiply and tf.matmul



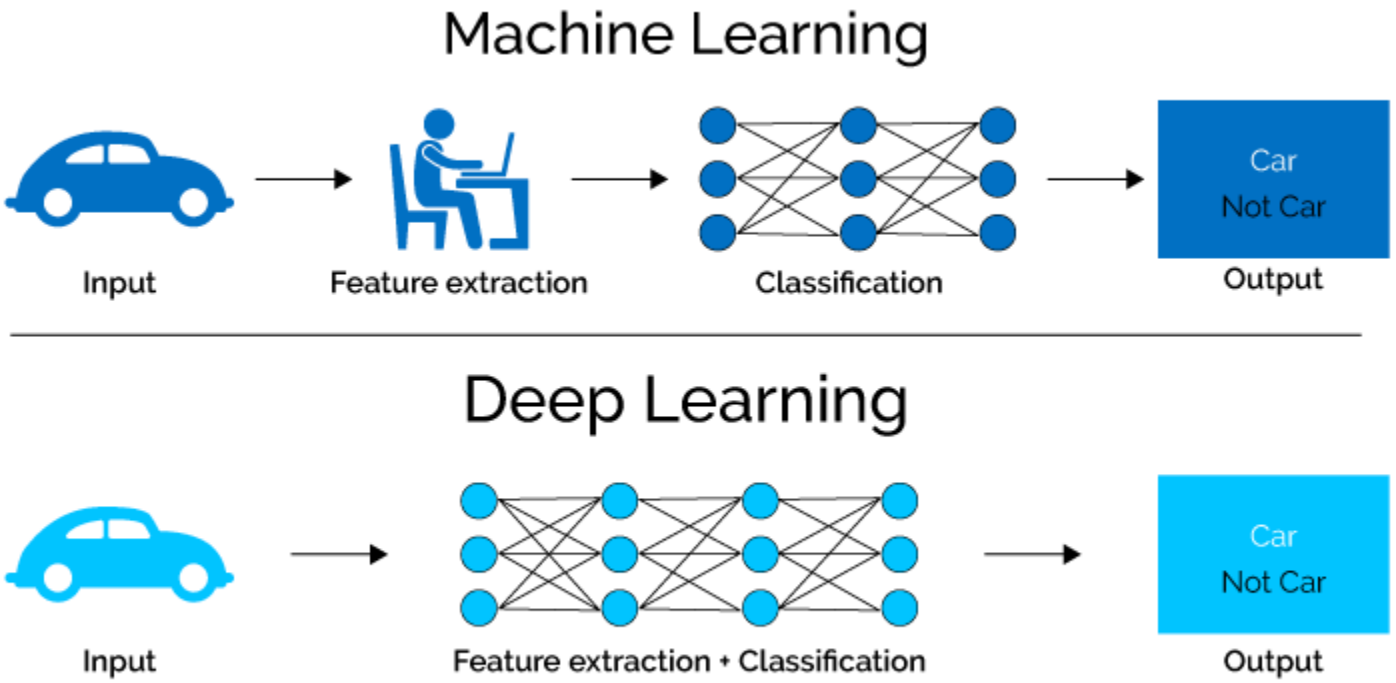
Dot Product

Matrix Multiplication

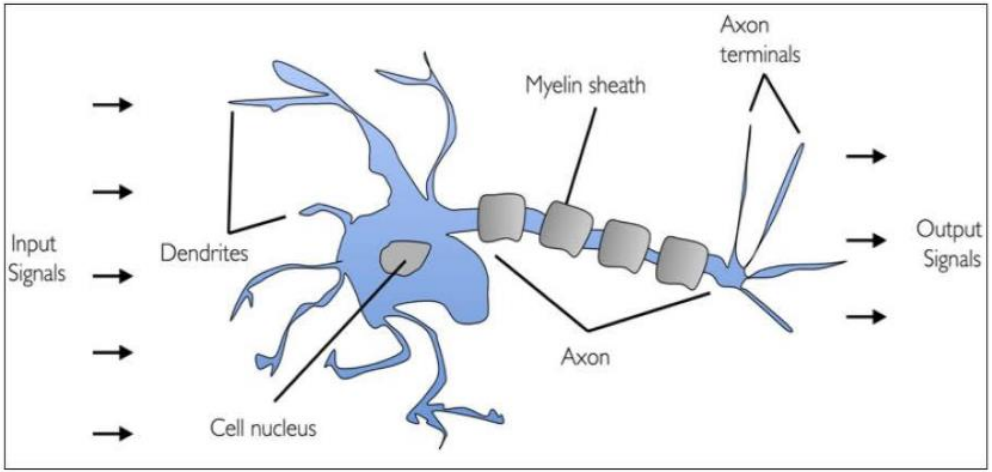
Deep Learning Vs Machine Learning

Factors	Deep Learning	Machine Learning
Data Requirement	Requires large data	Can train on lesser data
Accuracy	Provides high accuracy	Gives lesser accuracy
Training Time	Takes longer to train	Takes less time to train
Hardware Dependency	Requires GPU to train properly	Trains on CPU
Hyperparameter Tuning	Can be tuned in various different ways.	Limited tuning capabilities

What is the need of Deep Learning?

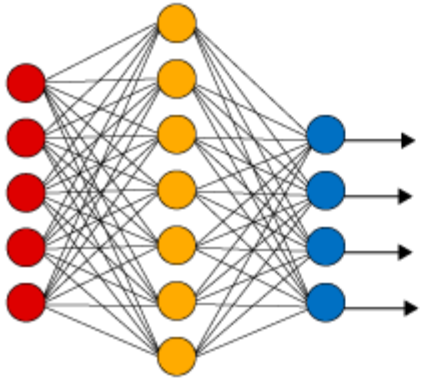


Biological Neuron

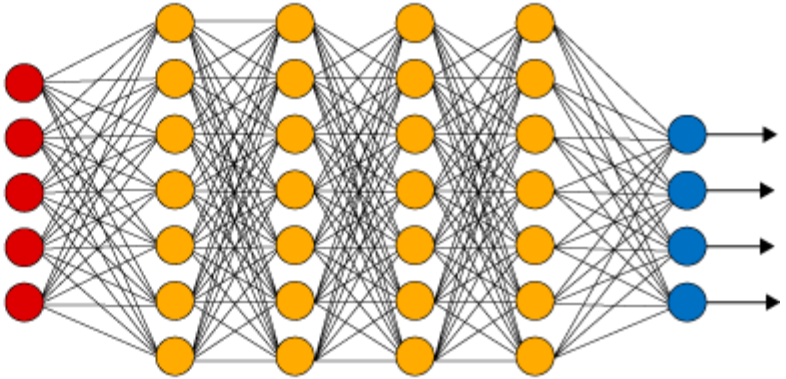


Biological Neuron	Artificial Neuron
Cell Nucleus (Soma)	Node
Dendrites	Input
Synapse	Weights or interconnections
Axon	Output

Simple Neural Network

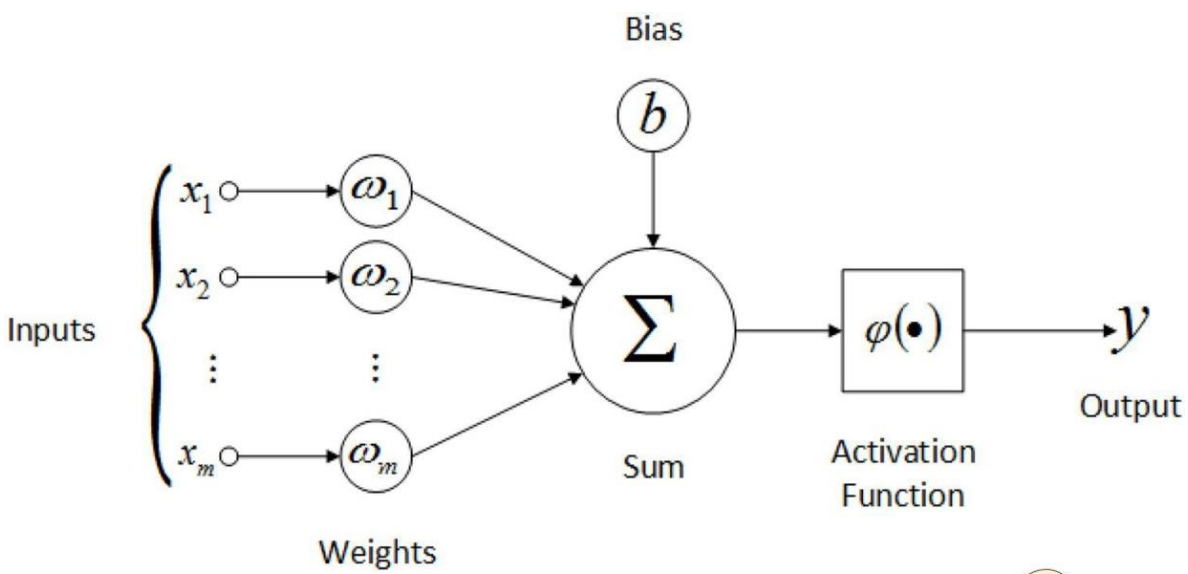


Deep Learning Neural Network



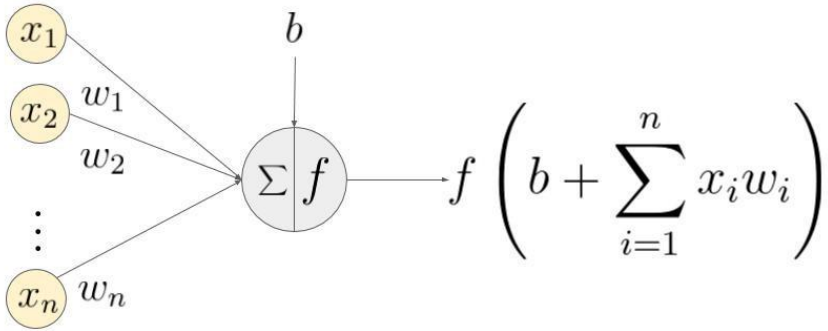
Input Layer Hidden Layer Output Layer

Artificial Neuron - Perceptron



1. Inputs
2. Weights
3. Bias
4. Activation Function
5. Output

Matrix Multiplication between Input values and Weights



Weights and Biases are trainable parameters

An example of a neuron showing the input (x_1 - x_n), their corresponding weights (w_1 - w_n), a bias (b) and the activation function f applied to the weighted sum of the inputs.

Why we need bias in Neural Network?

The bias unit guarantees that even when all the inputs are zeros there will still be an activation in the neuron.

Keras has implemented some functions for getting or setting weights for every layer.

- **layer.get_weights()**: returns the weights of the layer as a list of Numpy arrays.
- **layer.set_weights(weights)**: sets the weights of the layer from a list of Numpy arrays.

`get_weights()` for a `Dense` layer returns a list of two elements, the first element contains the weights, and the second element contains the biases. So you can simply do:

```
weights = model.layers[0].get_weights()[0]
biases = model.layers[0].get_weights()[1]
```

Note that weights and biases are already numpy arrays.

Artificial Neural Network

1. Feedforward Network
2. Backpropogation



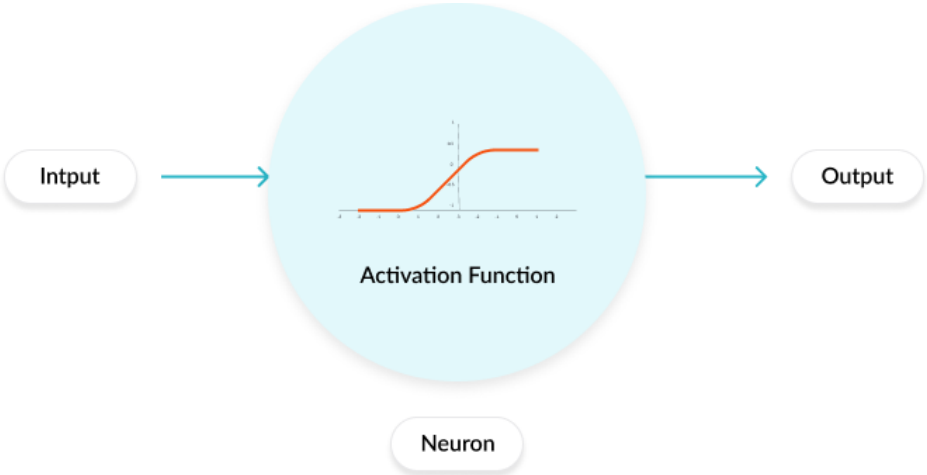
Head of Google Brain

Activation Function

- In neural networks, activation functions determine the output of a node from a given set of inputs

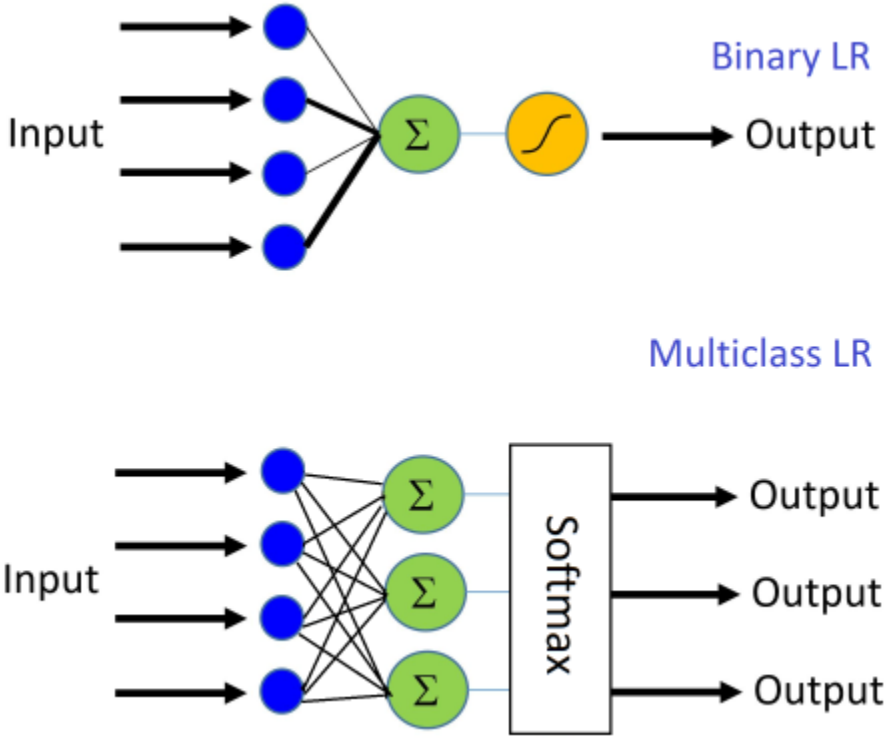
Some commonly used Activation Functions

- Sigmoid – Output Layer (Binary)
- Softmax(Multi Class)
- Relu – Hidden Layer
- Leaky ReLu
- Tanh
- Linear



https://dashee87.github.io/deep%20learning/visualising-activation-functions-in-neural-networks/

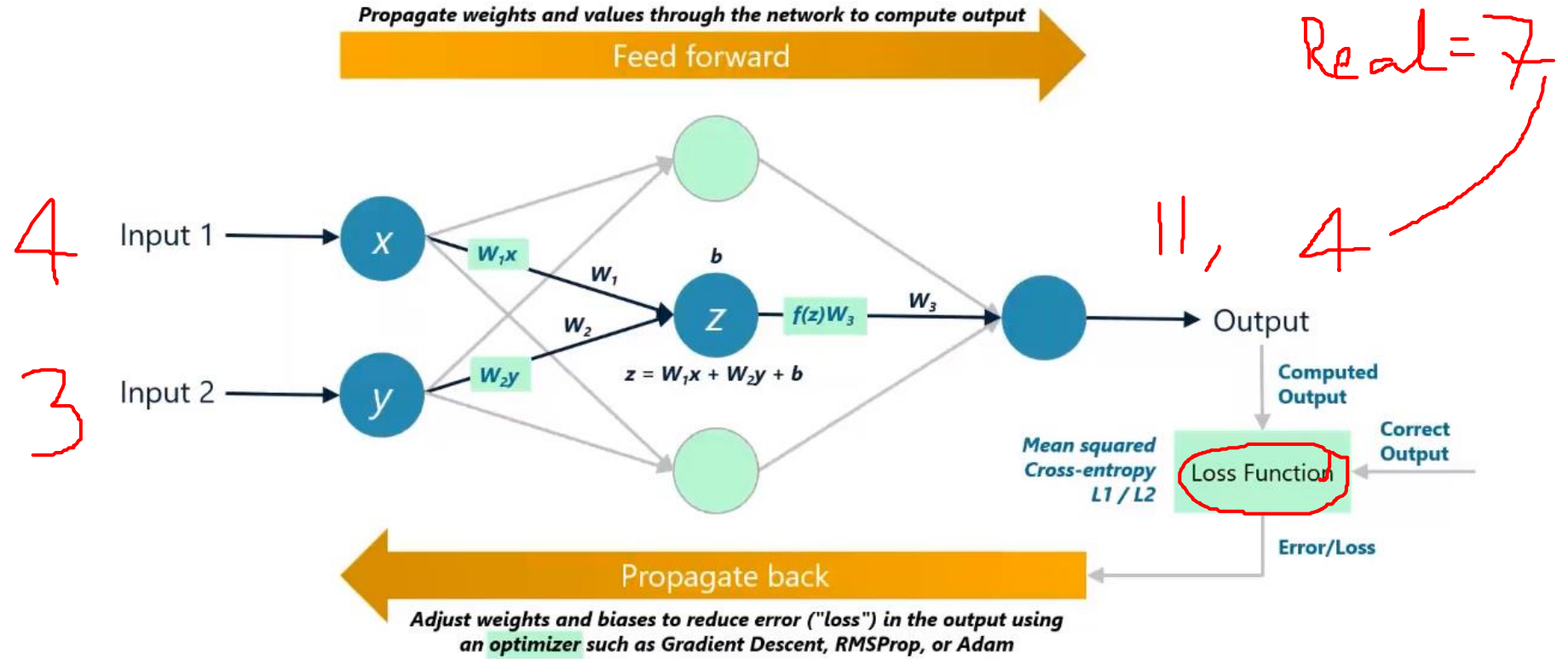
Softmax Activation



- It is a generalization of the logistic function to multiple dimensions

- It returns into a discrete probability distribution over the target classes

Backpropagation



Loss Function

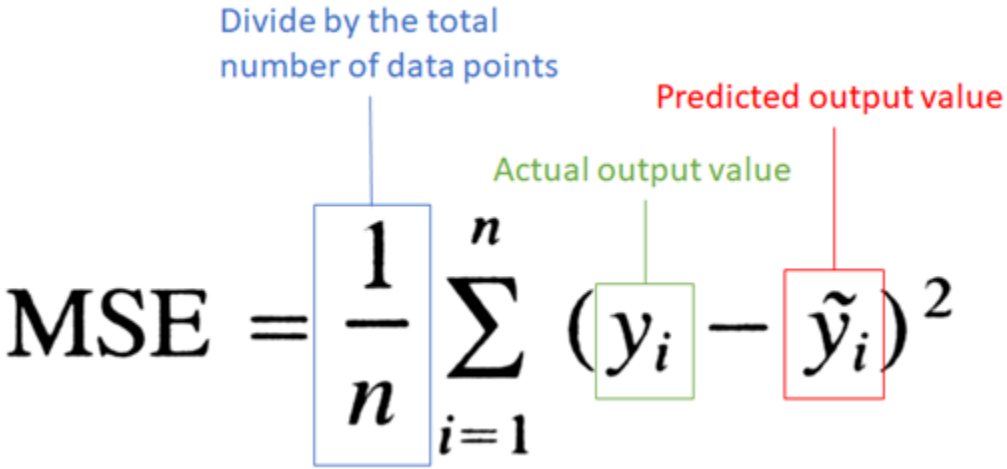
Error Function Also called Cost Function

A **function** that maps an event or values of one or more variables onto a real number intuitively representing some “loss" associated with the event. An optimization problem seeks to minimize a **loss function**.

1. **Regression Loss** – Mean Square Error, Mean Absolute Error
2. **Classification/Probabilistic Loss** – Binary Cross Entropy, Categorical, Sparse Categorical

https://ml-cheatsheet.readthedocs.io/en/latest/loss_functions.html

Mean Square Error



Cross Entropy Loss Function

1. **Categorical Cross Entropy** – Targets should be in One Hot Encoding Form
2. **Sparse Categorical Cross Entropy** – Targets can be in integer

- If your targets are **one-hot encoded**, use `categorical_crossentropy`.
  - Examples of one-hot encodings:
    - `[1,0,0]`
    - `[0,1,0]`
    - `[0,0,1]`
- But if your targets are **integers**, use `sparse_categorical_crossentropy`.
  - Examples of integer encodings (*for the sake of completion*):
    - `1`
    - `2`
    - `3`

https://stackoverflow.com/questions/58565394/what-is-the-difference-between-sparse-categorical-crossentropy-and-categorical-c

Deciding number of Neurons

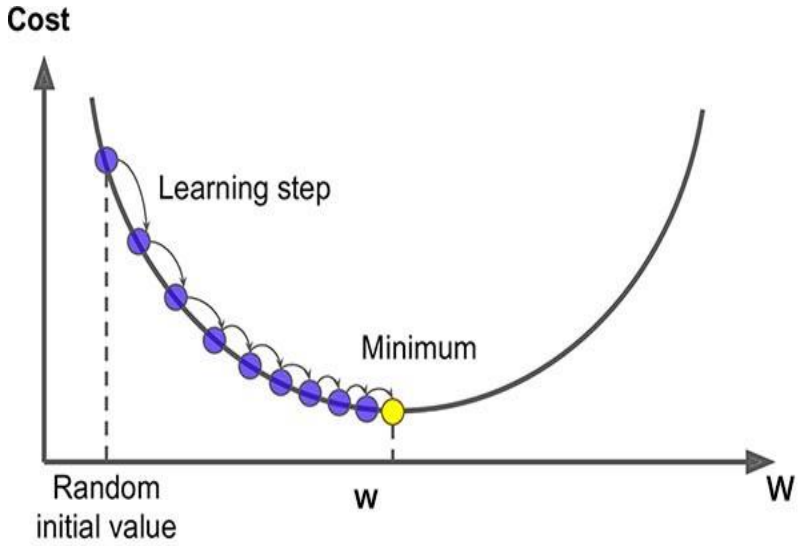
There are many rule-of-thumb methods for determining the correct number of neurons to use in the hidden layers, such as the following:

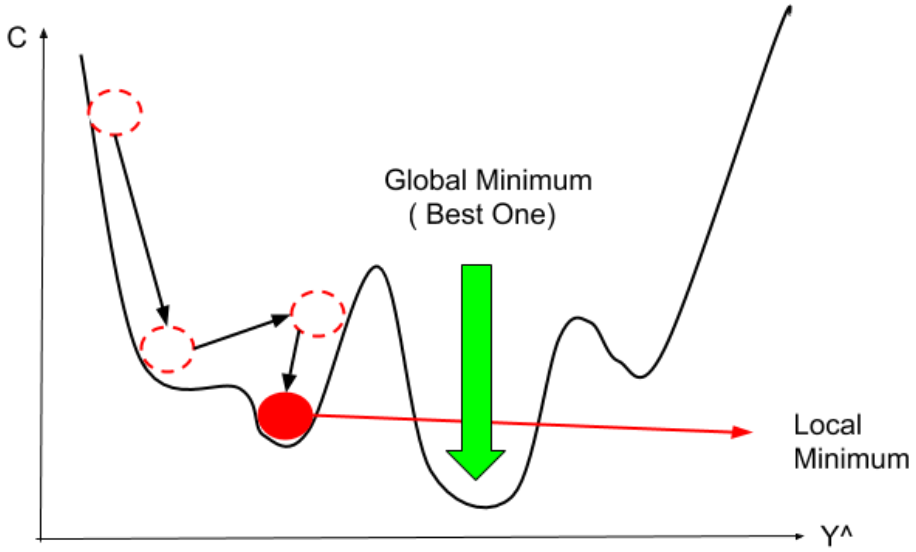
- The number of hidden neurons should be between the size of the input layer and the size of the output layer.
- The number of hidden neurons should be 2/3 the size of the input layer, plus the size of the output layer.
- The number of hidden neurons should be less than twice the size of the input layer.

These three rules provide a starting point for you to consider. Ultimately, the selection of an architecture for your neural network will come down to trial and error.

https://keras-team.github.io/keras-tuner/

Weights vs Loss (Gradient Descent)

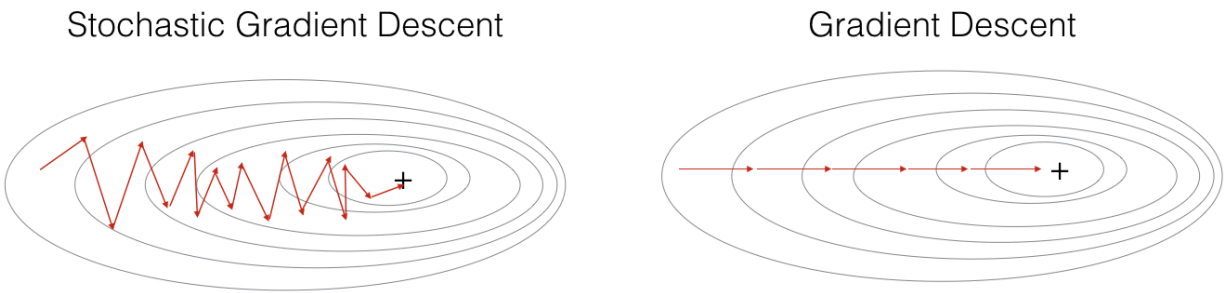




Optimizers

Here Learning Rate is fixed

1. Gradient Descent
2. Stochastic Gradient Decent
3. Mini Batch Stochastic Gradient Descent



SGD has Noise. We can reduce Noise by introducing momentum

https://snaildove.github.io/2018/03/02/02_optimization-algorithms/

4. Adagrad – Adaptive Gradient Descent
- Here the learning Rate dynamically changes

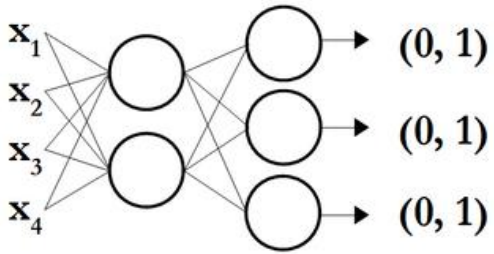
Adam is a popular algorithm in the field of deep learning because it achieves good results fast.

Deep Learning

How does the algorithm make a decision?



Feedforward



$$H_{in} = XW_1$$

$$H_{out} = \text{sigmoid}(H_{in})$$

$$O_{in} = H_{out}W_2$$

$$O_{out} = \text{sigmoid}(O_{in})$$

How do you determine the right parameters for the algorithm?



Cost function/Loss function:

$$\text{Mean Squared Error} = \frac{1}{2N}\sum_{e}\sum_{n}(O_{out\,e,n} - Y_{e,n})^2$$

Gradient Descent:

$$W_i := W_i - \alpha \cdot \frac{\partial\,\text{MSE}}{\partial\,W_i}(W_i)$$

Chain Rule:

$$\frac{\partial Z}{\partial X} = \frac{\partial Z}{\partial Y} \cdot \frac{\partial Y}{\partial X}$$

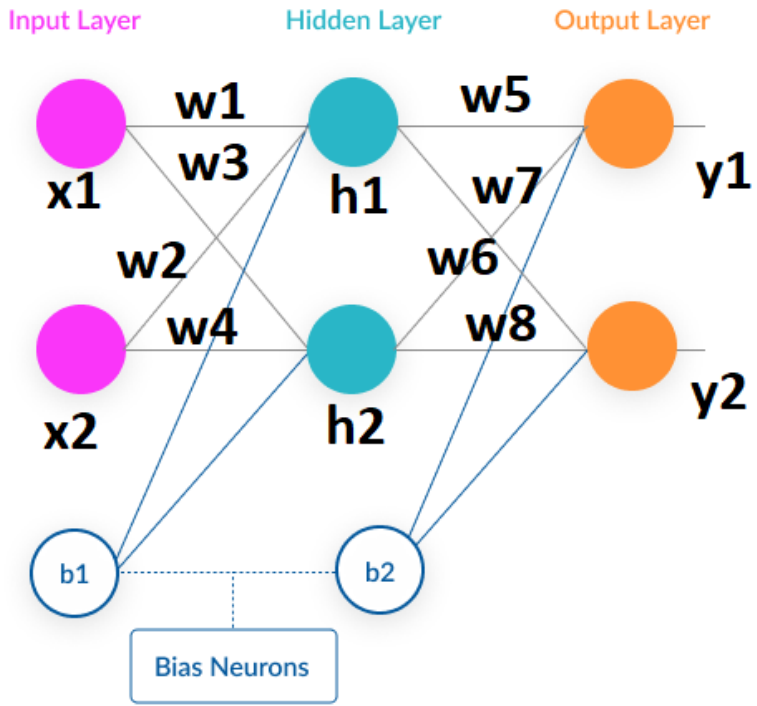
Backpropagation

$$\frac{\partial\,\text{MSE}}{\partial\,W_1}(W_1) = \frac{\partial\,\text{MSE}}{\partial\,O_{out}} \frac{\partial\,O_{out}}{\partial\,O_{in}} \frac{\partial\,O_{in}}{\partial\,H_{out}} \frac{\partial\,H_{out}}{\partial\,H_{in}} \frac{\partial\,H_{in}}{\partial\,W_1}$$

$$\frac{\partial\,\text{MSE}}{\partial\,W_2}(W_2) = \frac{\partial\,\text{MSE}}{\partial\,O_{out}} \frac{\partial\,O_{out}}{\partial\,O_{in}} \frac{\partial\,O_{in}}{\partial\,W_2}$$

N: e * n

Simple Neural Network



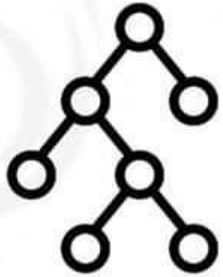
Epoch :

An Epoch represent one iteration over the entire dataset.



Batch :

We cannot pass the entire dataset into the Neural Network at once. So, we divide the dataset into number of batches.



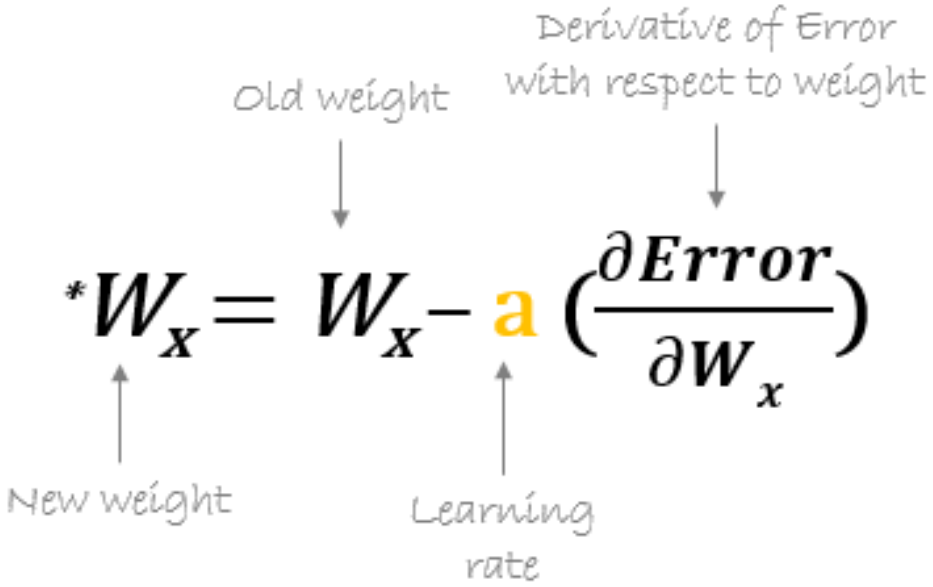
Iteration :

If we have 1000 images as Data ane a batch size of 20, then an Epoch should run 1000/20 = 50 iteration.



INFINITYCODEX

Backpropogation



For all 8 weights

$$w_1^+ = w_1 - \eta \frac{\partial E_{total}}{\partial w_1}$$

$$w_2^+ = w_2 - \eta \frac{\partial E_{total}}{\partial w_2}$$

$$w_3^+ = w_3 - \eta \frac{\partial E_{total}}{\partial w_3}$$

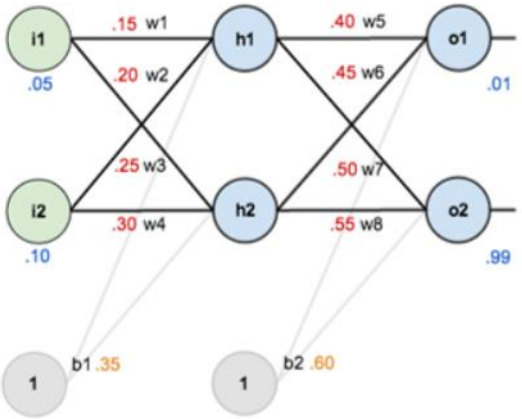
$$w_4^+ = w_4 - \eta \frac{\partial E_{total}}{\partial w_4}$$

$$w_5^+ = w_5 - \eta \frac{\partial E_{total}}{\partial w_5}$$

$$w_6^+ = w_6 - \eta \frac{\partial E_{total}}{\partial w_6}$$

$$w_7^+ = w_7 - \eta \frac{\partial E_{total}}{\partial w_7}$$

$$w_8^+ = w_8 - \eta \frac{\partial E_{total}}{\partial w_8}$$



Consider w_5. We want to know how much a change in w_5 affects the total error, aka $\frac{\partial E_{total}}{\partial w_5}$.

> $\frac{\partial E_{total}}{\partial w_5}$ is read as "the partial derivative of E_{total} with respect to w_5". You can also say "the gradient with respect to w_5".

By applying the chain rule we know that:

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Chain Rule

$$\frac{\partial Error}{\partial w} = \frac{\partial Error}{\partial outo} * \frac{\partial outo}{\partial ino} * \frac{\partial ino}{\partial w}$$

Figure 66: Calculate the derivative

Partial Derivative 1

$$\frac{\partial Error}{\partial outo} = (outo - target\ output)$$

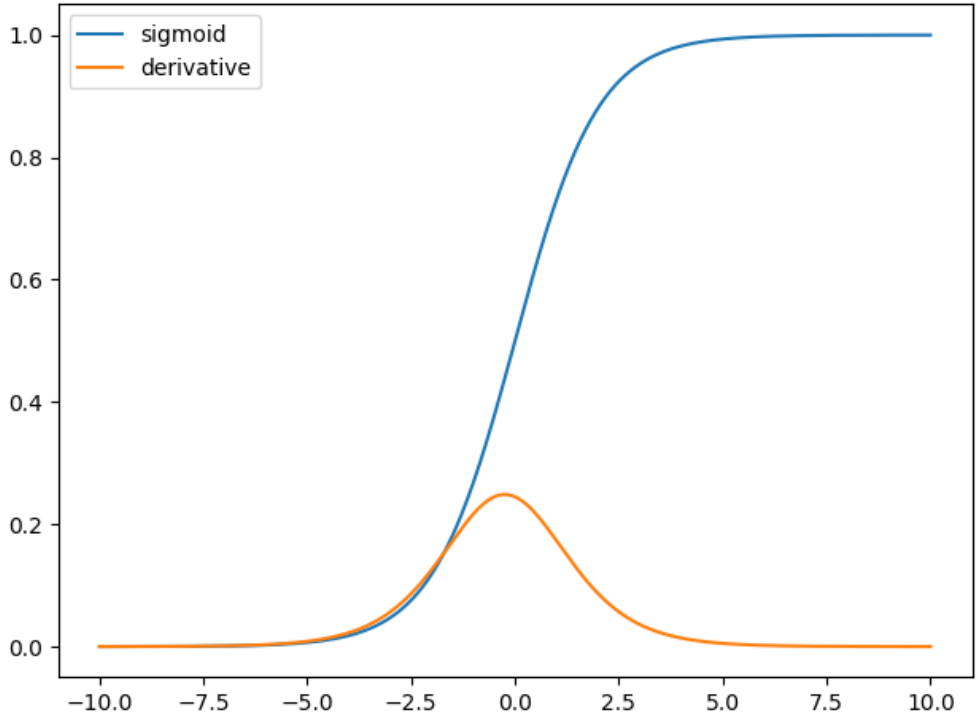
Figure 67: First derivative

Partial Derivative 2

$$\frac{\partial outo}{\partial ino} = outo\,(1 - outo)$$

Figure 68: Second derivative

Derivative of Sigmoid



$$\frac{ds(x)}{dx} = \frac{1}{1+e^{-x}}$$

$$= \left(\frac{1}{1+e^{-x}}\right)^2 \frac{d}{dx}(1+e^{-x})$$

$$= \left(\frac{1}{1+e^{-x}}\right)^2 e^{-x}(-1)$$

$$= \left(\frac{1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right)(-e^{-x})$$

$$= \left(\frac{1}{1+e^{-x}}\right)\left(\frac{-e^{-x}}{1+e^{-x}}\right)$$

$$= s(x)(1-s(x))$$

Partial Derivative 3

$$\frac{\partial ino}{\partial w} = input\ values$$

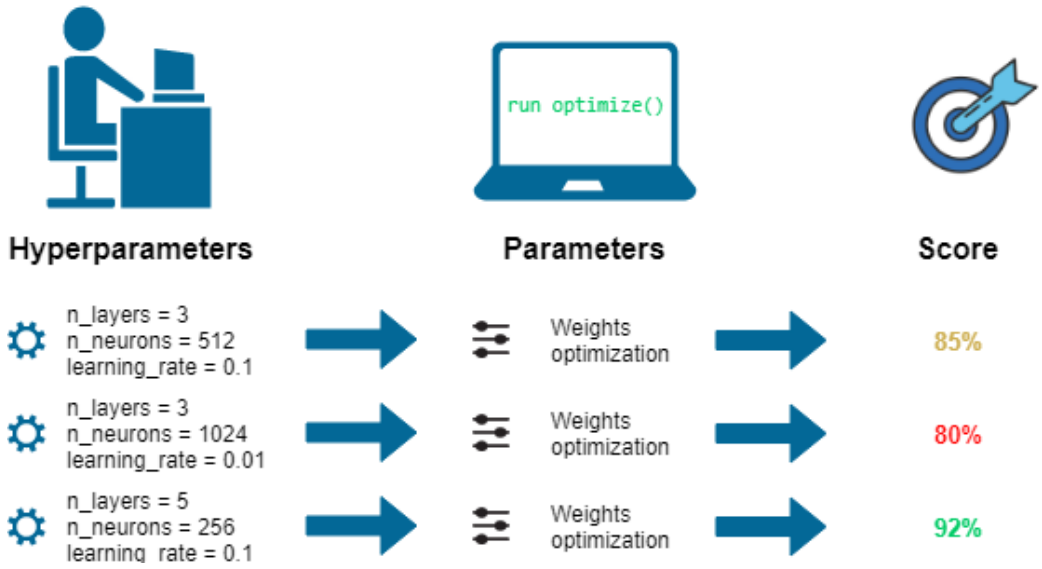
Figure 69: Third derivative

Visualizing ANN

- http://experiments.mostafa.io/public/ffbpann/

Hyperparameter Tuning

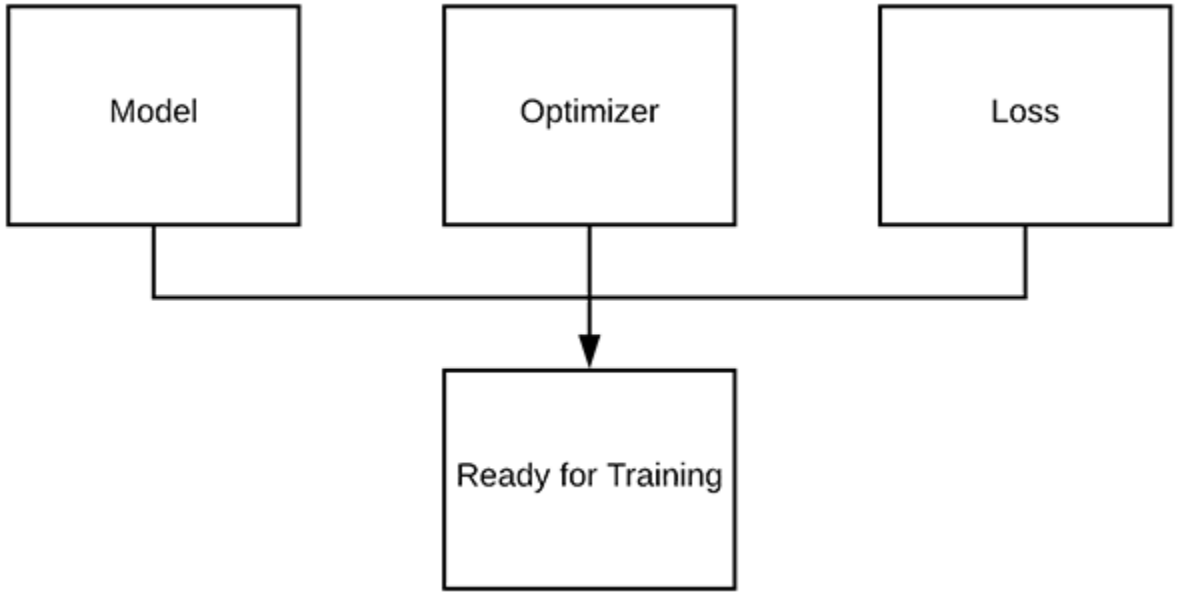
Hyperparameters are the variables that govern the training process and the topology of an ML model



Hyperparameters are of two types:

- **Model hyperparameters** which influence model selection such as the number and width of hidden layers
- **Algorithm hyperparameters** which influence the speed and quality of the learning algorithm such as the learning rate for Stochastic Gradient Descent (SGD)

https://keras-team.github.io/keras-tuner/


Model
Optimizer
Loss
Ready for Training

High Level vs Low Level TF

HIGH LEVEL APPROACH	LOW LEVEL APPROACH
1. We use Keras Model	1. We use Linear Algebra
2. Weights and Bias values are hidden	2. We should assign separately
3. Becomes easy for longer data	3. Code becomes complex